

RISC-V 技术文档 Demo

葛良晨

2026 年 5 月 26 日

目录

0.1	修订历史	iii
第一章	Roadmap of Linux Kernel	1
1.1	系统使用与管理	1
1.1.1	进程与服务管理	1
第二章	Nix 包管理	5
第三章	IC 设计相关知识积累	6
3.1	一些易用设置	6
第四章	InfiniBand 介绍	7
4.1	发展历程	7
4.2	核心特性	7
4.2.1	RDMA (远程直接内存访问)	7
4.2.2	高带宽与低延迟	8
4.2.3	可靠传输	8
4.3	协议层次	8
4.4	核心概念	8
4.4.1	队列对 (Queue Pair, QP)	8
4.4.2	完成队列 (Completion Queue, CQ)	8
4.4.3	保护域 (Protection Domain, PD)	9
4.4.4	内存注册 (Memory Registration, MR)	9
4.4.5	全局标识符 (GID) 与本地标识符 (LID)	9
4.5	硬件组件	9
4.5.1	HCA (Host Channel Adapter)	9
4.5.2	交换机	9
4.5.3	子网管理器 (Subnet Manager, SM)	9
4.5.4	线缆与连接器	9
4.6	与以太网/RoCE 的对比	10
4.7	InfiniBand 在 AI 训练中的应用	10
4.8	关键术语速查	10
4.9	Verbs API 编程概览	11
第五章	Hermes Agent 相关知识	12
5.1	rsync 文件同步工具	12
5.1.1	常用选项	12
5.1.2	Hermes 数据同步中的应用	12

5.1.3	与 cp/scp 的对比	12
5.2	LLM 对话原理：上下文与“模拟对话”	12
5.2.1	核心认知	12
5.2.2	每次请求都发送完整历史	13
5.2.3	对话是模拟出来的	13
5.2.4	KV Cache 与 Prompt Caching	13
5.2.5	启示	14
5.3	Hermes Feishu 网关连接与用户白名单 Debug 记录	14
5.3.1	问题描述	14
5.3.2	环境	14
5.3.3	日志现象	14
5.3.4	原因分析	14
5.3.5	解决步骤	15
5.3.6	验证	16
5.3.7	关键文件路径	16
5.3.8	飞书相关环境变量	16
5.4	DeepSeek API 消费监控	16
第六章	个人日记	18
6.1	2026 年	18
6.1.1	5 月	18
附录 A	一些碎碎念	23

0.1 修订历史

1.0 葛良晨 (2025-05-4)

Added

- L^AT_EX 模版搭建完成

第一章 Roadmap of Linux Kernel

Linux 学习的 Roadmap，主要包括三大主题（系统的使用与管理，核心子系统原理，内核深度剖析），以及一个额外的高级主题。

- **系统的使用与管理**：这一部分不是简单介绍基础操作。而是比价深入的理解各种基础概念。
 - **进程与服务管理**：使用 ps/top/htop 等工具，相关的系统调用 fork/exec/wait 等；服务管理 systemd/units；Linux 的启动流程演化路径。
 - **用户/权限/安全模型**：UID/GID/权限位/ACL；sudo 机制，PAM（安全）capabilities（替代 root）；Linux 安全模块（LSM）和 SELinux/AppArmor 等安全框架；Linux 的安全加固和最佳实践。
 - **文件系统与存储管理**：VFS 抽象；常见 FS ext4/xfs/btrfs/procfs/sysfs；mount / namespace。存储上，block device；LVM；RAID；buffer vs cache；direct IO/ buffered IO；IO 调度器（CFQ/Deadline/NOOP）；
 - **网络管理**：socket 编程；TCP/IP；ss/netstat；iproute2 工具；iptables/nftables；network namespaces；
 - **系统监控与性能分析**：iostat/vmstat/pidstat；perf 工具；strace/ltrace/bpfttrace；systemtap/eBPF；日志管理（syslog/journald/dmesg）；
- **核心子系统原理**：这一部分深入探讨 Linux 内核的核心子系统，包括进程管理、内存管理、文件系统、网络协议栈等。通过理解这些子系统的原理，读者可以更好地理解 Linux 内核的工作机制。
- **内核深度剖析**：这一部分将深入分析 Linux 内核的实现细节，包括内核模块开发、调试技巧、性能优化等内容。通过实际的代码示例和案例分析，读者可以掌握内核开发和调试的技能。
- **高级主题**：这一部分将涵盖一些高级主题，如实时内核、嵌入式 Linux、安全性增强等。这些内容将帮助读者了解 Linux 在特定领域的应用和发展趋势。

1.1 系统使用与管理

1.1.1 进程与服务管理

启动流程

启动流程 Linux 的启动流程可以分为以下几个阶段：

- **BIOS/UEFI 阶段**：计算机开机后，BIOS 或 UEFI 固件会进行硬件初始化，并寻找可引导设备。找到后，加载引导加载程序（如 GRUB）到内存并执行
- **引导加载程序阶段**：引导加载程序负责加载 Linux 内核和初始 RAM 盘（initrd/initramfs）到内存，并将控制权转交给内核。

- **内核初始化阶段：**内核开始执行，进行硬件检测和初始化，挂载根文件系统，并启动第一个用户空间进程（通常是 `init` 或 `systemd`）。内核初始化完成后，内核会调用 `init` 进程，通常是通过 `execve("/sbin/init", ...)`，这样第一个用户态进程（PID 1）就开始运行。
- **用户空间初始化阶段：**`init` 或 `systemd` 负责启动系统服务和用户进程，完成系统的初始化。

展开说说内核的启动过程 Bootloader（比如 GRUB）完成的工作有：

- 加载内核映像（`vmlinuz`）和初始 RAM 盘（`initrd/initramfs`）到内存。
- 设置内核启动参数 `cmdline`（如 `root=`、`console=` 等）。
- 将控制权转交给内核，通常是通过跳转到内核入口点（如 `0x100000`）来实现。

下面就进入到内核启动流程：

1. **体系结构相关的初始化：**内核会进行 CPU 特定的初始化，建立最初的栈，如设置 GDT/IDT、启用分页、初始化中断控制器、设置 CPU 模式等。
2. **`start_kernel` 函数：**这是内核的主入口点，负责调用各种初始化函数来设置内核环境。
3. **内核子系统初始化：**内核会依次初始化内存管理、调度器、中断子系统、定时器、设备模型、驱动框架和文件系统等核心模块，并完成根文件系统的挂载准备。

为什么要挂载 `initramfs`？

如何创建 `kernel_thread` 的？内核 PID 1 是如何变成用户态的？

`initramfs->rootfs` 切换是如何进行的？

内核的概念 内核用于整个操作系统核心，用于管理资源（CPU，内存，网络，存储，系统调用等）。内核是资源的拥有者和仲裁者。指责包括：

- **进程调度：**分配 CPU 时间片，管理进程状态（就绪、运行、阻塞等）。
- **内存管理：**分配和回收内存，管理虚拟内存和物理内存。
- **文件系统：**提供统一的接口访问不同类型的文件系统。
- **网络协议栈：**处理网络通信，支持各种协议（TCP/IP 等）。
- **设备驱动：**管理硬件设备的访问和控制。
- **系统调用：**提供用户空间与内核空间的接口，允许用户程序请求内核服务。

内核的主要工作方式：事件驱动 + 被动执行 + 少量主动线程

内核的代码运行在 3 种 context 中：

- **用户空间进程：**当用户空间的程序执行时，它会通过系统调用进入内核态，内核会执行相应的内核代码来处理请求。
- **中断处理程序：**当硬件设备发出中断信号时，内核会暂停当前正在执行的代码，转而执行相应的中断处理程序来响应硬件事件。

- **内核线程**：内核可以创建自己的线程（kernel thread），这些线程在内核态运行，执行内核任务，如处理网络请求、管理内存等。

Listing 1.1: Linux 内核的核心子系统

Linux Kernel

- 进程调度（Scheduler）
- 内存管理（Memory Management）
- 虚拟文件系统（VFS）
- 设备驱动（Drivers）
- 网络子系统（Networking）
- 系统调用接口（Syscall）
- 安全机制（LSM / 权限）

内核中的执行体 假设一个单核 CPU，它每时每刻都在执行指令，那么这个指令来自于哪里呢？有这样几种可能：

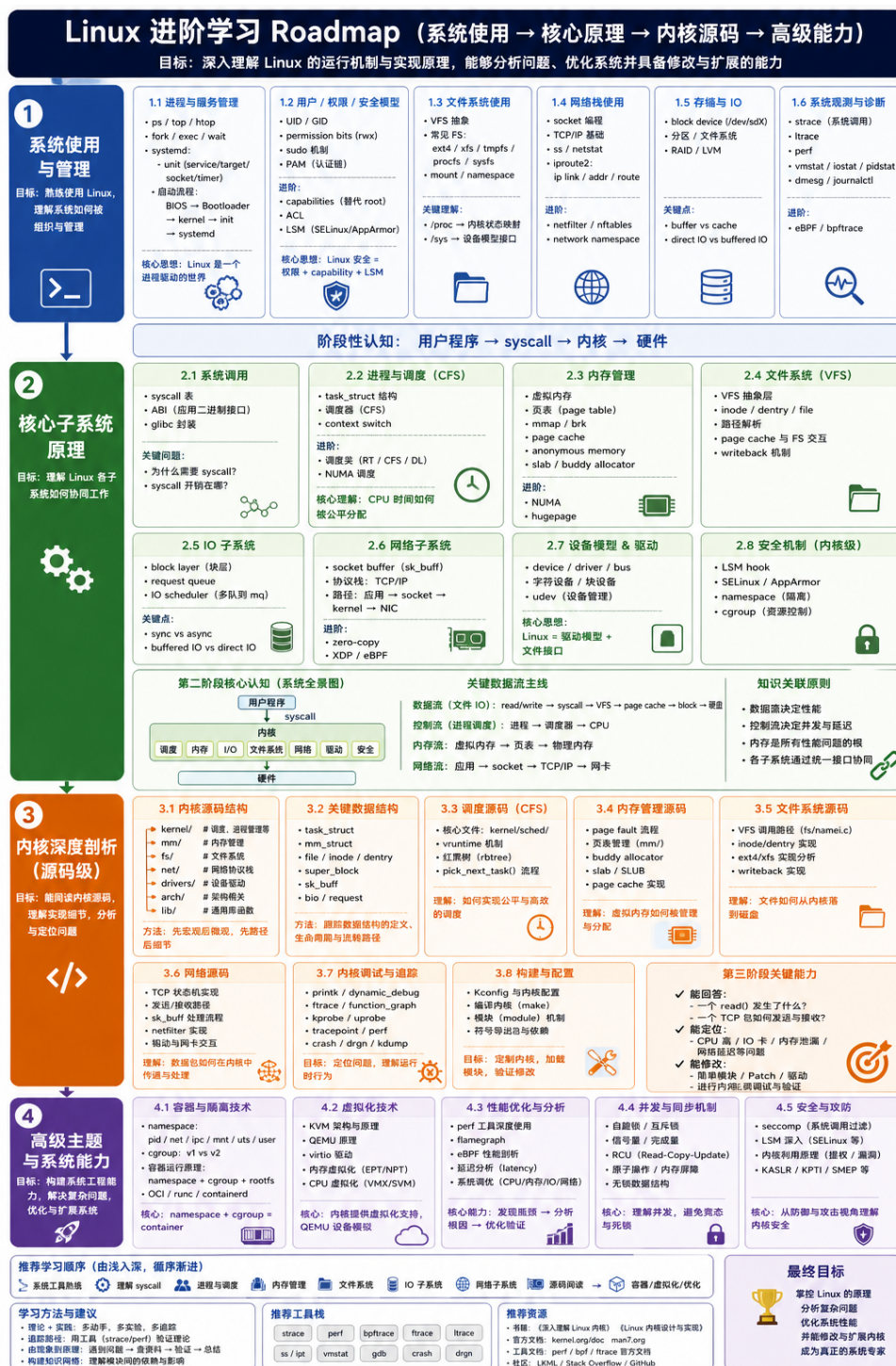
- **用户空间进程**：当用户空间的程序执行时，它会通过系统调用进入内核态，内核会执行相应的内核代码来处理请求。
- **内核 syscall**：当用户空间进程调用系统调用时，内核会执行相应的系统调用处理函数来完成用户请求。
- **内核线程**：内核可以创建自己的线程（kernel thread），这些线程在内核态运行，执行内核任务，如处理网络请求、管理内存等。
- **中断处理程序**：当硬件设备发出中断信号时，内核会暂停当前正在执行的代码，转而执行相应的中断处理程序来响应硬件事件。

现代 Linux 发行版已经从原本的 `init+etc/rc*.d` 脚本的启动方式 SysV，转变到了使用 `systemd`。启动模型也从原本串行执行脚本，变成依赖驱动的并行服务调度。

PID 1 的 `init` 进程是系统启动后第一个运行的用户空间进程，负责启动和管理其他系统服务和用户进程。它是系统的根进程，所有其他进程都是它的子孙进程。它具有如下的特点：

- **祖先进程**：所有其他进程都是 `init` 的子孙进程。当一个进程的父进程终止时，`init` 会接管这些孤儿进程。否则这些孤儿进程将成为僵尸进程，占用系统资源。
- **信号语义特殊**：`init` 进程对某些信号有特殊的处理方式。例如，`SIGTERM` 信号会被 `init` 忽略，而不是终止进程。这是为了确保系统的稳定性和可靠性。
- **系统初始化和服务管理**：`init` 负责启动和管理系统服务。比如，挂载为难系统，启动服务，设置环境。它会根据配置文件（如 `/etc/inittab` 或 `systemd` 的 `unit` 文件）来启动必要的服务，并确保它们在系统运行期间保持运行。

将内核和 PID 1 进行对比，重点是内核管理资源，PID 1 管理进程和服务。内核类似于政府 + 法律，PID 1 类似于总理，负责执行和管理系统服务，组织社会的运行。首相也不能违法，他在用户层，必须通过 `syscall` 来请求内核服务。



第二章 Nix 包管理

这个章节介绍了 Nix 包管理器的基本使用方法和一些常见的命令，同样也涉及了 NixOS 的相关内容。包含在 macOS 上使用 Nix 包管理器的说明。

第三章 IC 设计相关知识积累

Testbench 阶段标注 为了在 Verilog 中的仿真波形中看到目前测试的执行阶段，我们可以在 Testbench 中添加一些标志位或者输出语句来标注当前的阶段。例如：

```
typedef enum int {
    ST_INIT    = 0,
    ST_REG_RW  = 1,
    ST_SPLIT   = 2,
    ST_WSTRB   = 3,
    ST_STREAM  = 4,
    ST_DONE    = 5
} test_stage_e;
test_stage_e test_stage_id;

task automatic set_stage(input test_stage_e id, input string name);
begin
    test_stage_id = id;
    test_stage = name;
end
endtask
```

3.1 一些易用设置

Verdi 缩放波形居中显示 在使用 Verdi 进行仿真波形查看时，可以通过以下设置来缩放波形并居中显示：在 nWave 窗口中，选择 Waveform -> Keep Cursor at Center，这样在缩放波形时，光标会保持在中心位置，方便观察波形的细节。

第四章 InfiniBand 介绍

InfiniBand（简称 IB）是一种高性能互连技术，主要用于高性能计算（HPC）和数据中心。它提供了极高的带宽、极低的延迟，并原生支持 RDMA（Remote Direct Memory Access），是超级计算机和 AI 训练集群的事实标准互连方案。

4.1 发展历程

InfiniBand 诞生于 1999 年，由 Compaq、Dell、HP、IBM、Intel、Microsoft 和 Sun 等公司共同推动，目标是取代 PCI 总线，成为服务器内部与服务器之间的统一互连架构。虽然最终未能取代 PCI，但在 HPC 领域取得了巨大成功。

速率演进

代际	信号速率	有效带宽 (4x)	发布时间
SDR	2.5 Gbps/lane	10 Gbps	2001
DDR	5 Gbps/lane	20 Gbps	2005
QDR	10 Gbps/lane	40 Gbps	2007
FDR	14 Gbps/lane	56 Gbps	2011
EDR	25 Gbps/lane	100 Gbps	2014
HDR	50 Gbps/lane	200 Gbps	2017
HDR100	50 Gbps/lane	100 Gbps (2x)	2017
NDR	100 Gbps/lane	400 Gbps	2020
XDR	200 Gbps/lane	800 Gbps	2024
GDR	400 Gbps/lane	1600 Gbps	规划中

每代带宽翻倍，同时延迟持续降低。NDR 400 Gbps 已成为当前主流部署，XDR 800 Gbps 正在落地。

4.2 核心特性

4.2.1 RDMA（远程直接内存访问）

RDMA 是 InfiniBand 最核心的能力。它允许一台主机直接读写远程主机的内存，完全绕过远端 CPU 和内核网络栈。

- **零拷贝**：数据从网卡直接写入应用缓冲区，不经过内核缓冲区
- **内核旁路**：数据传输不经过内核协议栈，没有系统调用开销

- **CPU 卸载**：远端 CPU 完全不参与数据传输

一次 RDMA 操作的延迟通常在 1–2 μs 以内，而传统 TCP/IP 网络往返时延在 10–50 μs 。

4.2.2 高带宽与低延迟

InfiniBand 交换机采用 cut-through 转发模式，单跳交换延迟低至 90 ns 以下。结合 RDMA，端到端延迟可控制在微秒级。

4.2.3 可靠传输

InfiniBand 链路层提供基于信用的流控机制（Credit-based Flow Control），确保零丢包。传输层提供可靠连接（RC）、不可靠数据报（UD）等多种服务类型，应用可根据需求选择。

4.3 协议层次

InfiniBand 协议栈分为五层：

层次	功能	类比 TCP/IP
物理层	电气信号、连接器、线缆规范	物理层
链路层	包格式、流控、子网内路由	链路层
网络层	子网间路由（GRH 全局路由头）	IP 层
传输层	分段/重组、QP 管理、可靠传输	TCP/UDP
上层协议	SDP、SRP、IPoIB、RDS 等映射	应用层

4.4 核心概念

4.4.1 队列对（Queue Pair, QP）

QP 是 InfiniBand 通信的基本端点，由发送队列（SQ）和接收队列（RQ）组成。每个 QP 绑定一个本地端口，所有通信操作都基于 QP 进行。

QP 的服务类型：

- **RC (Reliable Connection)**：可靠连接，一对一，支持所有 RDMA 操作
- **UC (Unreliable Connection)**：不可靠连接，一对一，不保证送达
- **UD (Unreliable Datagram)**：不可靠数据报，一对多，支持多播
- **RD (Reliable Datagram)**：可靠数据报，一对多，QDR 后基本废弃
- **XRC (Extended Reliable Connected)**：扩展可靠连接，支持多对多
- **DCI (Dynamically Connected)**：动态连接，一对多共享接收端

4.4.2 完成队列（Completion Queue, CQ）

CQ 用于收集已完成的发送/接收操作信息。多个 QP 可以共享同一个 CQ，应用通过轮询（polling）CQ 来获取操作完成状态。

4.4.3 保护域 (Protection Domain, PD)

PD 是一个安全隔离域，定义哪些内存区域和 QP 可以相互访问。同一个 PD 内的 QP 才能操作该 PD 注册的内存。

4.4.4 内存注册 (Memory Registration, MR)

RDMA 操作前，应用需要把本地内存缓冲区注册到 HCA，获得 lkey（本地密钥）和 rkey（远端密钥）。远端主机需要 rkey 才能访问该内存。这一步是为了将物理内存页钉住（pin），防止被换出。

4.4.5 全局标识符 (GID) 与本地标识符 (LID)

- **LID**: 16 位本地地址，由子网管理器 (SM) 分配，仅在子网内有效
- **GID**: 128 位全局地址（类 IPv6），用于跨子网路由

4.5 硬件组件

4.5.1 HCA (Host Channel Adapter)

HCA 即 IB 网卡，一端接 PCIe 总线与主机内存相连，一端接 IB 线缆与交换机相连。HCA 负责所有 IB 协议处理（包括传输层、链路层），将 CPU 从网络处理中解放出来。

主流供应商：NVIDIA（原 Mellanox）、Intel（原 QLogic，已退出）。

4.5.2 交换机

IB 交换机执行简单的 LID 查表转发，单跳延迟极低。IB 网络不需要像以太网那样运行生成树 (STP)，路由由 SM 统一计算并下发给所有交换机。

4.5.3 子网管理器 (Subnet Manager, SM)

SM 是 InfiniBand 子网的核心控制面，负责：

- 发现拓扑并扫描所有节点
- 为每个端口分配 LID
- 计算子网内的转发路径 (Forwarding Tables)
- 维护子网运行状态

SM 可以运行在交换机（硬件 SM）上，也可以运行在服务器（软件 SM，如 OpenSM）上。

4.5.4 线缆与连接器

类型	读写速率	典型距离
铜缆 (DAC)	到 400 Gbps	3–5 m
有源光缆 (AOC)	到 400 Gbps	30–100 m
光模块 (QSFP)	到 400 Gbps	100 m–10 km

4.6 与以太网/RoCE 的对比

RoCE (RDMA over Converged Ethernet) 是在以太网上实现 RDMA 的技术，分为两个版本：

- **RoCEv1**：以太网链路层直接承载 IB 传输层包，仅限同二层网络
- **RoCEv2**：将 IB 传输层封装在 UDP/IP 中，支持 IP 路由

维度	InfiniBand	RoCEv2
网络层	IB 子网 + GRH	IP/UDP
流控	基于信用 (Credit-based)，无损	依赖 PFC/ECN，配置复杂
拥塞控制	原生支持，由 SM/FM 管理	需要 DCQCN 等算法辅助
管理复杂度	SM 集中管理，即插即用	需要 PFC/ECN/DCB 全网调优
生态	NVIDIA 封闭生态，成本高	多厂商支持，成本低
延迟	极低 (1μs)	略高 (依赖交换机)
丢包	零丢包	PFC 配置不当可能出现
路由	SM 集中计算，确定性路由	标准 IP 路由
最大规模	已部署超 40000 节点	理论上无限制

选型建议：极致性能和超大规模 HPC/AI 训练集群选 InfiniBand；接入层、多厂商混合、成本敏感场景选 RoCEv2。

4.7 InfiniBand 在 AI 训练中的应用

当前主流 AI 训练集群 (如 NVIDIA SuperPOD) 广泛使用 InfiniBand 互连：

- GPU 节点通过 8 块 HCA 连接到胖树 (Fat-Tree) IB 交换机网络
- 使用 NCCL 库实现 GPU 间直接 RDMA 通信，AllReduce 操作可完全卸载到网络
- SHARP (Scalable Hierarchical Aggregation and Reduction Protocol) 在交换机内完成归约运算，减少网络流量
- 节点数万卡规模的训练，InfiniBand 是目前唯一经过大规模验证的互连方案

4.8 关键术语速查

缩写	全称 / 含义
HCA	Host Channel Adapter (IB 网卡)
TCA	Target Channel Adapter (存储/IO 端 HCA)
QP	Queue Pair (通信端点, SQ + RQ)
CQ	Completion Queue (完成通知队列)
PD	Protection Domain (保护域)
MR	Memory Region (注册的内存缓冲区)
MW	Memory Window (MR 的子窗口, 用于动态权限)

缩写	全称 / 含义
SM	Subnet Manager (子网管理器)
MAD	Management Datagram (管理报文)
LID	Local Identifier (子网内地址)
GID	Global Identifier (全局地址)
GRH	Global Routing Header (全局路由头)
MTU	Maximum Transfer Unit (最大传输单元)
RC	Reliable Connection
UC	Unreliable Connection
UD	Unreliable Datagram
RDMA	Remote Direct Memory Access
RC QP	可靠连接 QP, 最常用
SRQ	Shared Receive Queue (多个 QP 共享接收队列)
PKey	Partition Key (分区隔离)
SHARP	交换机内聚合/归约加速
NCCL	NVIDIA Collective Communications Library

4.9 Verbs API 编程概览

InfiniBand 的编程接口称为 Verbs，核心流程如下：

1. 获取设备列表 `ibv_get_device_list()`
2. 打开设备 `ibv_open_device()`
3. 分配保护域 `ibv_alloc_pd()`
4. 注册内存 `ibv_reg_mr()`
5. 创建完成队列 `ibv_create_cq()`
6. 创建队列对 `ibv_create_qp()`
7. 修改 QP 状态 `ibv_modify_qp()`: RESET → INIT → RTR → RTS
8. 连接交换：通过 TCP 带外通道交换 QP 信息 (QPN、LID、GID、密钥等)
9. 发起 RDMA 操作: `ibv_post_send()` 提交 WR 到 SQ
10. 轮询完成: `ibv_poll_cq()` 等待 WC (Work Completion)

RDMA 操作类型：

- **SEND/RECV**: 类似传统消息收发，接收端需预置接收缓冲区
- **RDMA WRITE**: 单端写入远端内存，无需远端 CPU 参与
- **RDMA READ**: 单端读取远端内存
- **ATOMIC**: 原子的 CAS/Fetch-and-Add，用于分布式锁和同步

第五章 Hermes Agent 相关知识

5.1 rsync 文件同步工具

rsync (remote sync) 是 Linux/macOS 上的文件同步工具，核心特点是**增量复制**——只传输有差异的部分，而非每次都全量拷贝。

5.1.1 常用选项

选项	含义
-a	archive 模式：递归 + 保留权限/时间/属主/符号链接等
--delete	删除目标端存在但源端已不存在的文件
-v	verbose，显示详细信息
-z	传输时压缩
--dry-run	模拟运行，不实际复制

5.1.2 Hermes 数据同步中的应用

在 Hermes 数据同步脚本中，其用法如下：

```
rsync -a --delete "$HERMES_DIR/skills/" "$REPO_DIR/skills/"
```

这表示：递归同步 `~/.hermes/skills/` 到仓库的 `skills/` 目录，保持文件属性不变，并删除仓库里本地已被删除的多余文件。

5.1.3 与 cp/scp 的对比

- **cp**：每次都全量复制，不论文件是否变更
- **scp**：跨网络全量复制，不支持断点续传
- **rsync**：仅传输差异部分，支持增量、断点续传

简言之，rsync 是智能版的 cp/备份工具。

5.2 LLM 对话原理：上下文与“模拟对话”

5.2.1 核心认知

LLM（大语言模型）与生俱来的能力只有一项：给定一段文本，预测下一个最合理的 **token**。所谓的“对话”、“记忆”、“上下文理解”全都是 prompt engineering 包装出来的产物。

5.2.2 每次请求都发送完整历史

每次用户发一条消息，Hermes（或其他 LLM 客户端）做的事：

```
用户输入: "给我配飞书"

| Hermes 组装 prompt

[System Prompt (角色设定)]
[环境信息 (OS, CWD, ...)]
[持久化记忆 (user preferences)]
[历史对话 (压缩后)]
- 用户: 配置飞书
- 助手: 查日志 -> 发现 lark-oapi 缺失 -> 安装
- 用户: 删掉错误 user
- 助手: 改 .env -> 重启
[工具调用结果]
[本轮新消息: "给我配飞书"]

| 发送给 LLM API (几十万 token)

LLM 做文本接龙 -> 返回回复
```

用户只发了几个字节。客户端（Hermes）拼了几十万 token 发给 API。

5.2.3 对话是模拟出来的

你以为的	实际发生的
模型在“和你对话”	模型在做文本接龙
模型“记得”你刚才说了什么	那段文本就在输入里
对话是原生模式	对话是 prompt 格式的产物
模型有“意图”	模型只是在模仿训练数据里的对话模式

关键创新是 **ChatML 格式**（由 OpenAI 提出）：

```
user: 法国的首都哪里?
assistant: 法国的首都是巴黎。
user: 那日本呢?
assistant: [LLM 在这里做文本接龙]
```

模型在训练时见过大量这种格式，学会了在这种格式下应该如何“扮演”。

5.2.4 KV Cache 与 Prompt Caching

每次发消息确实需要重新处理整个 prompt，但有两个优化：

KV Cache（推理引擎层面）

- **Prefill 阶段**：把完整 prompt 一次性输入，计算所有 token 的 KV cache（主要计算瓶颈）
- **Decode 阶段**：每吐一个 token，只需要增量计算最后一个 token 的 KV cache，复用前面的

Prompt Caching (API 层面)

- 无缓存：每次重新算全部历史的 KV，全部按 input token 计费
- 有缓存：重复部分按约 10% 缓存费率，新内容按全价

DeepSeek、Claude、Gemini、OpenAI 等主流模型均支持。

5.2.5 启示

1. 对话不是 LLM 的原生能力，而是通过数据格式和 prompt 设计模拟出来的
2. 每次请求都发送完整历史（包括所有历史对话 + system prompt + 环境信息）
3. 所谓“记忆力好”只是因为每次的输入里都包含了全部历史文本
4. 用户端的流量很小（几字节），但到 API 的流量很大（几十万 token）

5.3 Hermes Feishu 网关连接与用户白名单 Debug 记录

5.3.1 问题描述

Hermes Agent 飞书（Feishu/Lark）网关启动后无法回复用户消息。

- 在飞书上发消息给机器人，没有收到任何回复
- Gateway 日志中显示飞书适配器未加载

5.3.2 环境

- macOS 15.6
- Hermes Agent v0.13.0 (Nix 安装)
- Python 3.12.13 (Nix 环境)
- 飞书连接模式：WebSocket
- Feishu App ID: cli_aa8f26395378dbdb

5.3.3 日志现象

```
WARNING gateway.run: Feishu: lark-oapi not installed or FEISHU_APP_ID/SECRET not set
WARNING gateway.run: No adapter available for feishu
```

5.3.4 原因分析

原因 1: lark-oapi 库未安装

Hermes Gateway 使用 Nix 环境中的 Python 3.12(路径: `/nix/store/...-hermes-agent-env/bin/python`), 而 `lark-oapi` 库不存在于该环境中。飞书适配器在 `import lark_oapi` 失败时将 `FEISHU_AVAILABLE` 置为 `False`, 导致适配器无法启动。

原因 2: launchd 缺少 PYTHONPATH

即使安装了库, Gateway 的 launchd plist (`~/Library/LaunchAgents/ai.hermes.gateway.plist`) 中没有设置 `PYTHONPATH` 环境变量, Python 找不到额外安装的包。

原因 3: 用户白名单限制

`.env` 中 `FEISHU_ALLOW_ALL_USERS=false`, 初始白名单 `FEISHU_ALLOWED_USERS=b5g9a7g9` 中没有包含实际发消息的用户。日志中表现为:

```
WARNING gateway.run: Unauthorized user: ou_xxx (None) on feishu
```

5.3.5 解决步骤

1. 安装 lark-oapi 到 Nix Python 可访问的路径

由于 Nix 环境是只读的, `lark-oapi` 无法通过 `pip install` 直接安装到 Nix store 中的 `site-packages`。采用以下方案:

```
# 使用 macOS 系统 Python 的 pip 安装到 ~/.hermes/site-packages/
# 指定 Python 3.12 编译的 wheel 以保证兼容性
python3 -m pip install \
  --target ~/.hermes/site-packages \
  --only-binary=:all: \
  --python-version 3.12 \
  --platform macosx_11_0_arm64 \
  --abi cp312 \
  lark-oapi
```

2. 在 launchd plist 中添加 PYTHONPATH

编辑 `~/Library/LaunchAgents/ai.hermes.gateway.plist`, 在 `<key>EnvironmentVariables</key>` 的 `<dict>` 中添加:

```
<key>PYTHONPATH</key>
<string>/Users/katsu/.hermes/site-packages</string>
```

然后重新加载 plist:

```
launchctl unload ~/Library/LaunchAgents/ai.hermes.gateway.plist
launchctl load ~/Library/LaunchAgents/ai.hermes.gateway.plist
```

3. 更新用户白名单

在 `.env` 中找到 `FEISHU_ALLOWED_USERS`, 将新用户的 `open_id` 追加到逗号分隔的列表中:

```
FEISHU_ALLOWED_USERS=ou_8f29366d98158fbc2b52c7772dba32de
```

然后重启网关使其生效:

```
launchctl stop ai.hermes.gateway
sleep 2
launchctl start ai.hermes.gateway
```

5.3.6 验证

重启后日志显示飞书正常连接：

```
INFO gateway.run: feishu connected
[Lark] [INFO] connected to wss://msg-frontier.feishu.cn/ws/v2?...
```

5.3.7 关键文件路径

文件	用途
~/hermes/.env	Feishu 配置 (APP_ID, APP_SECRET, 白名单等)
~/hermes/site-packages/	手动安装的 lark-oapi 库 (Nix Python 通过 PYTHONPATH)
~/Library/LaunchAgents/ai.hermes.gateway.plist	Gateway 的 launchd 服务配置
~/hermes/logs/gateway.log	网关运行日志

5.3.8 飞书相关环境变量

变量	说明
FEISHU_APP_ID	飞书应用 ID (cli_xxx)
FEISHU_APP_SECRET	飞书应用密钥
FEISHU_DOMAIN	域名, 国内飞书填 feishu, 国际版填 lark
FEISHU_CONNECTION_MODE	连接模式: websocket 或 webhook
FEISHU_ALLOW_ALL_USERS	是否允许所有用户 (true/false)
FEISHU_ALLOWED_USERS	白名单, 逗号分隔的 open_id 列表
FEISHU_GROUP_POLICY	群组策略: open (所有群)、allowlist (仅白名单群)
FEISHU_REQUIRE_MENTION	是否要求 @ 提及才回复 (默认 true)

5.4 DeepSeek API 消费监控

脚本位置 hermes/scripts/deepseek-monitor.py

用途 查询 DeepSeek API 账户余额, 支持阈值告警和 Token 用量统计。

前提条件

- DEEPSEEK_API_KEY 环境变量已设置
- pip install requests

用法

仅查询余额

```
./hermes/scripts/deepseek-monitor.py
```

余额低于 10 CNY 时退出码为 2 (可用于告警链)

```
./hermes/scripts/deepseek-monitor.py --alert 10
```

```
# 从 Hermes session 日志累计 Token 用量
./hermes/scripts/deepseek-monitor.py --track ~/.hermes/sessions
```

API 端点

- GET <https://api.deepseek.com/user/balance>
- 官方文档: <https://api-docs.deepseek.com/zh-cn/api/get-user-balance>

输出示例

DeepSeek API 消费监控

状态: ✔ 可用
总余额: 55.07 CNY
充值余额: 55.07 CNY
赠金余额: 0.00 CNY
查询时间: 2026-05-15 15:34:08

集成到 Hermes cron 可用 `no_agent=true` 模式定时运行:

- 每天 9 点推送余额到飞书
- 余额低于 10 元时告警

第六章 个人日记

6.1 2026 年

6.1.1 5 月

5 月 15 日周五

1. **SPI 模块挂载**: 在 AXI-lite 总线上完成 SPI 控制器的挂载, 并编写了对应的 C 驱动程序。
2. **UART Monitor**: 完成了一个仅用于仿真环境的 UART monitor 模块。
3. **Hermes 个人系统构建**: 搭建了基于 Hermes Agent 的个人工作流系统, 配置了若干 cron 定时任务, 分别用于同步 Nix 知识库以及 Hermes 记忆与配置。

昨日被导师陈鑫严厉批评, 指摘我不把 *deadline* 当回事, 甚至放出狠话「是不是找打」。警醒之至, 当引以为戒。时间管理不可再松弛, 任务节点必须死守。

5 月 16 日周六

今天花了一天研究三个 ADC 校准算法的文档资料, 借助 AI 辅助理解算法原理和实现思路。后续计划写个 C 程序验证校准逻辑, 再制作一份 PPT 用于汇报展示——毕竟没有实际硬件环境, 仿真验证就够了。周一还得把代码导入到内网开发环境。

- **ADC 校准算法研究**: 仔细阅读了三份校准文档, 理解各算法的数学原理和适用场景。AI 辅助翻译和解释公式推导, 帮助厘清算法间的差异。
- **后续计划**: 编写 C 验证程序 + 制作 PPT, 周一导入内网。
- 李一航计划明天去看房子, 准备租房事宜。
- 月底需要审车 (车辆年检), 驾照 8 月份也到期要换证了——记得提前预约体检和照相。
- 天气盛美 🌞🌸, 就是风有点大。

今天感受:

早上老陈 9 点多就开始催进度

twemojigrinning face with sweat, 紧迫感拉满。不过研究下来发现 ADC 校准的思路逐渐清晰, 心里还算有底。得抓紧时间出代码和 PPT, 千万不能再拖了。

5 月 18 日周一

1. **组会汇报：本地大模型部署**：汇报了近期在本地大模型部署方面的进展，包括 Hermes Agent 系统搭建、模型推理和离线部署方案。
2. **CPU 校准算法讨论**：讨论了是否适合使用 SPI 方式做实时校准。**结论**：SPI 实时校准方案不太可行，这条路暂告一段落。校准算法后续可能走 **ASIC 路线**。
3. **后续计划**：原定今天找陈旭明老师讨论大模型本地部署方案，但最终未能进行。

5 月 19 日周二

1. **智谱企业咨询**：在智谱清言上进行了企业咨询，考虑部署 **GLM-5.1**，看起来是个不错的选择（适合本地部署场景）。
2. **Astera Labs 调研**：整理了 Astera Labs 这家公司的相关信息和产品线。该公司专注于数据中心互联（CXL 和 PCIe 衍生芯片），产品线涵盖 Retimer、Smart Memory Bridge 等。
3. **明日计划**：明天需向老陈汇报 Astera Labs 的调研成果，分析其可借鉴之处，以及对于本初创公司而言是否存在有挑战性的生态位——有什么可以切入的方向。¹

5 月 20 日周三

今天的主要任务是准备明天汇报用的材料。之前调研了 Astera Labs 的思路，需要整理成 PPT 向潘老师他们做汇报。

- **PPT 制作**：将 Astera Labs 的调研成果、公司技术路线和产品线整理成 PPT，同时梳理了自身项目的进展和下一步计划。给明天向潘老师的汇报做准备。
- 内容上主要围绕：公司背景调研成果、技术方案比较、以及后续研发方向的初步构想。

5 月 21 日周四

今天正式向潘老师等人进行了汇报。

- **汇报情况**：做了完整的 PPT 汇报，但效果不理想。
- **陈老师批评**：被陈老师批评了——他认为我完全应该从用户/听众的角度来组织和呈现汇报内容，而不是只从自己的视角去堆砌材料和数据。听众关心的是“这对我有什么价值”，而不是“我做了什么”。
- **反思**：这个批评很到位。做技术汇报时容易陷入“展示工作量”的误区，而忽略了听众的认知路径和需求。以后做任何汇报，都应该先想清楚：听众是谁？他们想听到什么？什么信息对他们有用？

5 月 22 日周五

今天做了一些关于 AI 硬件接口互联方面的调研，聚焦于芯片设计领域，看看有什么可以尝试的创业方向。

- **芯片设计方向调研**：调研了 AI 加速芯片领域的硬件接口互联方案，重点围绕**内存墙（Memory Wall）**问题展开分析——计算速度增长远快于内存带宽，导致大量算力被闲置等待数据搬运。

¹Astera Labs 的核心赛道是 CXL 内存互联和 PCIe 信号完整性，属于数据中心基础设施层。值得思考的是：是否有类似的高价值细分领域是我们有能力切入的？

- **创业方向探索**：从内存墙问题出发，思考是否存在值得切入的赛道——比如存算一体架构、HBM 接口方案、近存计算、CXL 内存池化等方向。对于本初创公司而言，内存墙是 AI 基础设施层的核心瓶颈，恰好是我们技术栈可能有所作为的领域。
- 整体还在初级阶段，需要进一步明确技术路线和市场切入点。

5 月 23 日周六

今天研究了一下用 Blender + Godot 做田园风格小游戏的开发方案。借助 AI 辅助做了些初步尝试和学习，感觉这条路还挺有意思的。

- **Blender + Godot 游戏开发尝试**：探索了 Blender 建模 + Godot 引擎的组合方案，目标是一个田园风格的小游戏。借助 AI 辅助完成了初步的学习和尝试——从模型导出、场景搭建到 Godot 中的脚本逻辑都有涉及。
- 3D 建模 + 轻量级游戏引擎的组合很适合个人开发者，后续想继续深入试试。

5 月 24 日周日

今天事情还挺多的，记录一下。

- **卖摩托车**：把摩托车卖掉了，成交价 3000 元。下周三才能办过户手续。
- **看房**：上午 10 点去看了租的房子，应该是在考虑搬家的事情。
- **午饭**：和李一航在附近的盒马吃了顿饭。
- **修手机**：下午去了龙湖天街的 vivo 官方维修店，换了屏幕总成和后盖，花了 540 元。

5 月 25 日周一

个人总结

今天刷抖音看到一位神经科学博主的分享，深受启发，总结了三条值得执行的原则：

1. 晨间禁屏，守住大脑的「可编程窗口」

起床后一小时内不要碰手机。早起时的大脑处于可编程状态，潜意识格外活跃，此时刷手机无异于给大脑注射了一剂高多巴胺阈值，使得全天都在渴望这种即时刺激。必须立下规矩：早上起床一小时内不碰手机；晚上睡觉也不躺着刷手机。

2. 对着自己说好话，语言即是暗示

念念不忘，必有回响——消极的心理暗示是自我实现的预言。一个长期被语言打压的孩子必然长不好，而我们不要做那个打压自己的老师或家长。每天对自己说鼓励的话，积极乐观。心理暗示的力量，可能远比我们想象的要大。

3. 干净饮食，喂给大脑好原料

所吃之物极大影响身体状态。要戒除高糖、高盐、高油的饮食，走向清淡和高蛋白，给大脑提供一个良好的物质基础。干净的输入决定干净的输出。

5 月 26 日周二

待办事项

1. 汇报 PPT 撰写：整理内网部署方案的价值，向潘老师做一次正式的展示。重点阐述——本地部署在数据安全、延迟控制、定制化能力和长期成本上的优势，以及落地路线图。
2. 产品线深度调研：围绕 AI 互联应用，走通从「芯片到产品」的完整链路。调研方向包括芯片互联方案（CXL / PCIe Retimer / 内存池化等）向具体产品的转化路径——哪些技术值得自研封装，哪些适合做市场切入点，最终形成清晰的产品定位。

个人总结

今天读到一篇公众号文章《程序员的那些事》，分享了 SpaceX 星舰 V3 首秀成功以及一张广为流传的梗图，感触颇深。

SpaceX 星舰 V3 首秀 & 工程哲学 5 月 23 日，马斯克旗下 SpaceX 星舰 V3 搭载猛禽第三代发动机（Raptor 3）首秀成功。随之诞生的是一张刷屏的梗图——

「先做出来 vs 之后再把它做好」



图 6.1: Raptor 3 引擎对比：干净利落的成熟形态

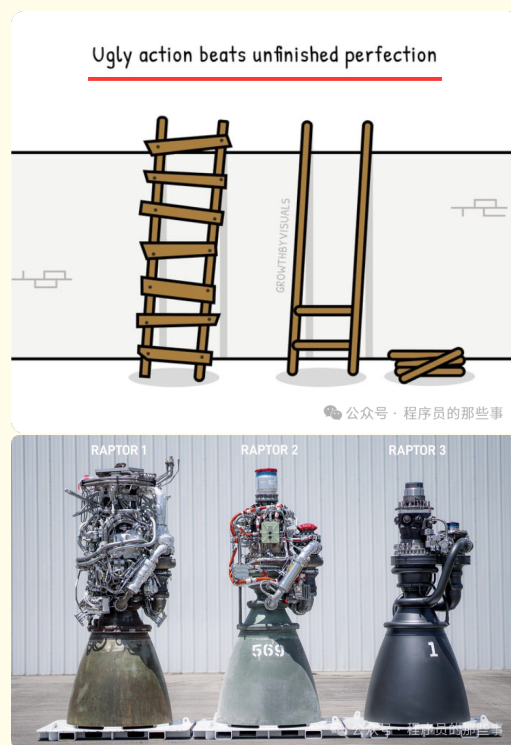


图 6.2: 星舰 V3 首秀

这个梗图直接把工程圈的底层逻辑说透了：

1. 先解决「从 0 到 1」的问题——星舰早期的发动机和助推器，核心目标是「能点火、能装上去、能飞起来」，哪怕管线外露、结构臃肿也没关系，先让系统跑通，拿到真实的测试数据。
2. 再通过迭代优化「从 1 到 100」——后续通过不断的测试，优化发动机设计、整合管线、提升集成度，最终变成那种「干净利落」的成熟形态。



图 6.3: 梗图：先做出来 vs 之后再把它做好

今天早上马斯克转发了这个梗图。有人说「真正难的不是做好，而是迈出第一步」。有人感慨人生里的所有事都适用。还有程序员说这就是软件工程里「重构」一词的最好图解：先写出能跑的代码，再慢慢打磨优雅的架构。

说到底，这张梗图戳中了很多人的痛点：我们总困在「完美主义陷阱」里不敢行动，却忘了

「你永远无法优化一个不存在的东西。」

不管是做产品、写代码，还是搞创作，**先做出一个能用的版本，再慢慢打磨**，才是最高效的路径。与自己的完美主义倾向做斗争，迈出第一步，Get shit done。

附录 A 一些碎碎念

i3wm 和 aerospace 是真的好用。窗口最大化直接通过快捷键完成，两个窗口并列操作也可以迅速完成。切换不同的工作区也很迅速，太完美了。简单易用，易上手，减少了许多鼠标的操作，不同费劲的用鼠标拖拽和调整大小，perfect!

真不错!